CAT-IA · Experiencia 2: agentes, herramientas y memoria

Autoras: Antonella Cuvertino y Miriam Hammami.

Docente: Sebastián Ignacio Sánchez Morales. Actualización: 11/10/2026. La fecha de entrega de EA2 y la sección están pendientes de confirmar; 09/09/2026 corresponde a EA1.

Qué cambió

EA1 analizaba cada ticket por separado mediante un flujo RAG fijo. EA2 incorpora una sesión persistente, una ventana de conversación, un resumen extractivo y herramientas tipadas. La interfaz permite elegir agente individual o equipo de tres roles. Conserva documentos, prioridad por reglas, citas y revisión humana.

La referencia recibida para esta implementación es el texto de RA2 e IL2.1–IL2.4 adjuntado por la estudiante. No se inventan ponderaciones ni aprobación del docente. Las decisiones de adaptación al equipo local se explicitan a continuación.

IL2.1 · Herramientas

catia/tools.py construye herramientas con el decorador tool de langchain_core. Cada solicitud tiene un ToolContext propio: no comparte citas, resultados ni memoria con otras sesiones.

- tool_search_knowledge_base(query, top_k): recupera fragmentos internos y externos respetando filtro, umbral y presupuesto.
- tool_check_sla_policy(impact, service_down, security_incident): calcula prioridad y primera respuesta. Los argumentos deben coincidir con los campos ingresados por el operador.
- tool_draft_response(): llama al generador, valida Decision, comprueba citas contra los fragmentos de esta solicitud y fuerza needs_review=true. Sin evidencia no llama al generador.

La firma de SLA conserva los indicadores existentes, en lugar de una urgencia textual ambigua. La herramienta de borrador toma evidencia del estado validado, no de una lista fabricada por el planificador. Son adaptaciones deliberadas de las firmas orientativas recibidas.

El catálogo contiene funciones de aplicación; no integra Gmail, Microsoft, un servicio de ticketera ni MCP. El generador consulta Ollama por HTTP y la búsqueda usa las fuentes externas de Mozilla incorporadas al corpus. No se afirma que sean llamadas web a Mozilla en tiempo real. La validación de citas sigue sin demostrar fidelidad semántica.

IL2.2 · Memoria corta y persistencia

catia/memory.py usa SQLite, incluido en Python. La ruta por defecto es runtime/memory.sqlite3 y está fuera del ZIP/Git. Una sesión tiene UUID, resumen, ventana de intercambios, contador y fecha de actualización UTC. La interfaz guarda únicamente el UUID en localStorage.

La ventana conserva por defecto tres intercambios redactados. Al superar ese número, la entrada humana más antigua se convierte en un extracto de hasta 240 caracteres que se incorpora a un resumen limitado a 900. La ventana y el resumen sobreviven a reiniciar el proceso. No se conserva el historial completo indefinidamente y no se entrena al modelo.

El resumen es extractivo y determinista, no un resumen generado por otra IA. Evita una llamada costosa al modelo local y puede perder matices o texto antiguo por truncamiento. Se usa la alternativa de ventana por session_id permitida en las instrucciones; no se depende de las clases históricas ConversationBufferWindowMemory/ConversationSummaryMemory.

El contexto entregado al generador incluye como máximo dos entradas previas recortadas y el resumen. La consulta RAG añade la última entrada previa al texto actual cuando el operador activa el contexto. Los borradores previos se muestran en pantalla, pero no se usan como evidencia ni se añaden a la consulta. Si cambia el tema, desactivar contexto o borrar el hilo. No hay detección semántica automática fiable de cambio de tema.

La misma sesión se serializa mediante locks del proceso. SQLite realiza las escrituras. Este diseño se verifica con un solo proceso de aplicación; no es una garantía de aislamiento transaccional de todo el flujo para varios workers. El UUID identifica el hilo, no autentica a una persona: el prototipo sigue siendo local, sin cuentas ni permisos multiusuario.

La memoria persiste texto con redacción parcial de patrones sensibles, a diferencia de las trazas, que siguen omitiendo el ticket y el borrador. Deben usarse datos ficticios. DELETE elimina la ventana y resumen locales, sin borrar archivos JSON descargados ni copias de seguridad externas. No se afirma anonimización completa ni borrado forense.

IL2.3 · Agente individual

catia/orchestration.py implementa un ciclo ReAct acotado sobre herramientas LangChain. Observa el ticket, selecciona una herramienta disponible, la ejecuta y vuelve a evaluar precondiciones. Con Ollama, una salida JSON restringida elige si empezar por búsqueda o política; cuando solo queda una acción válida, el programa la ejecuta directamente para ahorrar llamadas. El borrador siempre va después de búsqueda y política.

Hay un máximo de tres herramientas distintas por solicitud, lista permitida, contratos y rechazo de duplicados. No es un agente de autonomía abierta ni un planificador que puede ejecutar cualquier comando. La interfaz expone actor, herramienta, estado, duración y observaciones resumidas; no publica pensamientos internos del modelo.

En modo demo, la elección es determinista y se etiqueta langchain-react-demo. En modo real, se etiqueta langchain-react-ollama. La generación sigue usando Ollama y el esquema dinámico de citas. El conteo usage de la respuesta corresponde a la generación final; no agrega automáticamente el consumo del planificador ni todos los intercambios de CrewAI.

IL2.3 · Equipo secuencial

El segundo modo utiliza CrewAI real: tres objetos Agent con role, goal y backstory, tres Task con context que incluye las tareas anteriores y Process.sequential. Cada rol dispone de una herramienta mínima: Triage consulta SLA; Investigador RAG recupera evidencia; Redactor genera la categoría y el borrador validado.

Para reducir inferencias redundantes, la clasificación definitiva se realiza junto con la redacción; Triage se concentra en el impacto y la política. Esta distribución se diferencia del ejemplo que proponía clasificar también en Triage. No se presentan dos clasificaciones independientes ni se atribuye esa decisión al agente de reglas SLA.

LocalStageLLM adapta el protocolo textual de herramientas de CrewAI al servidor local. En demo responde de manera determinista para ejercitar el framework sin LLM. En modo real consulta Ollama antes de ejecutar la herramienta usando JSON restringido a la acción de ese rol; valida el nombre y lo traduce a Action/Action Input. El rol tiene una sola acción autorizada, por lo que esa llamada no representa una elección autónoma entre alternativas. El texto libre inicial falló en la integración con el modelo pequeño; el contrato restringido evita depender de que reproduzca el formato de CrewAI. Tras una observación validada, la conclusión de la etapa se compone a partir del resultado de herramienta; el texto libre del agente no reemplaza el contrato final.

Se limita cada agente a dos iteraciones y cada adaptador a tres llamadas, con tiempo máximo por petición. No hay delegación ni memoria nativa de CrewAI: se usa la memoria controlada de CAT-IA. La telemetría/tracing se desactiva. Las dependencias de CrewAI son mayores y se instalan mediante instalar_ea2.bat; la instalación base permite el modo individual.

Fallos y revisión humana

EA2 añade una opción visible para permitir simulación si falla el proveedor. Está desactivada por defecto. Si el operador la habilita y se produce un error de proveedor, se ejecuta el flujo individual con DemoGenerator. El resultado indica fallback_used=true, requested_provider=ollama, mode=demo, estado fallback_demo y advertencia. No se presenta como éxito del LLM ni como ejecución satisfactoria del equipo real solicitado.

Sin autorización de simulación se devuelve abstención explícita. Una salida inválida o dependencia ausente no se disfraza como simulación exitosa. La API continúa respondiendo con estado de negocio. Todos los borradores y abstenciones mantienen needs_review=true y human_approval_required=true. No existe botón de envío automático a clientes.

API y configuración

- POST /api/sessions crea un hilo y devuelve session_id.
- GET /api/sessions/{session_id} devuelve ventana, resumen y contador del hilo local.
- DELETE /api/sessions/{session_id} elimina su memoria.
- POST /api/tickets/analyze conserva los campos anteriores y admite session_id, orchestration=react/crew, use_context y allow_demo_fallback. Si no se proporciona sesión, crea una

nueva.

- GET /api/health declara memoria habilitada, modo por defecto y disponibilidad de CrewAI, además del modelo y corpus. No prueba inferencia real.

CATIA_ORCHESTRATION selecciona react o crew. CATIA_MEMORY_WINDOW controla N entre 1 y 8. CATIA_MEMORY_DB permite cambiar el archivo SQLite. CATIA_PROVIDER elige demo u ollama; OLLAMA_URL debe coincidir con el puerto del servicio. No se necesitan credenciales cloud para estos modos.

EA2 usa prompts/v4.txt para añadir continuidad sin reescribir las versiones históricas. El baseline Agent y scripts.evaluate conservan experimentos EA1 con su selección de versión. En /api/health la configuración RAG puede mantener v3 como baseline; el resultado EA2 informa v4 como versión efectivamente utilizada. La evaluación EA2 se ejecuta con scripts.smoke_ea2 y sus reportes separados.

IL2.4 · Arquitectura y archivos

docs/arquitectura.mmd representa el flujo actual. docs/arquitectura_ea1.mmd conserva el diagrama anterior. deliverables/arquitectura_EA2.pdf y presentacion_EA2_CAT-IA.pdf muestran la arquitectura y explicación de EA2. La presentación anterior se conserva como EA1. El informe y los resultados se reúnen en dossier_EA2_CAT-IA.pdf; el guion se conserva también separado para practicar.

ConversationAgent extiende el baseline en catia/agent.py. Las rutas se conectan en catia/api.py; los contratos en catia/schemas.py. Las nuevas responsabilidades están separadas en memory.py, tools.py y orchestration.py. prompts/v4.txt delimita los antecedentes como datos no confiables. tests/test_ea2.py verifica los nuevos comportamientos.

Pruebas y evidencia

reports/tests_ea2.xml y reports/coverage_ea2.json registran la suite EA1+EA2 ejecutada. Los casos incluyen continuidad, aislamiento, ventana, resumen, persistencia al reabrir, borrado, concurrencia de un hilo, redacción, precondiciones, argumentos, lista permitida, planificación alternativa, fallos y ciclo de sesiones API.

La prueba CrewAI con DemoGenerator ejecuta el framework y las herramientas reales usando decisiones simuladas; no acredita la calidad del modelo. scripts.smoke_ea2 --provider ollama --mode react/crew registra inferencia real separada si la ejecución termina sin fallback. El informe EA2 generado lee esos resultados y muestra estados efectivos, incluidos los fallos. No se trasladan los 20 aciertos de v3 a v4.

La fidelidad y relevancia humanas siguen pendientes. Una prueba de integración exitosa no demuestra generalización ni cumplimiento de toda una pauta aún no entregada íntegramente. El desempeño en CPU, el resumen extractivo y la ausencia de autenticación siguen siendo limitaciones.

Referencias de implementación

LangChain, herramientas tipadas y decorador: https://reference.langchain.com/python/langchain-core/tools/convert/tool. CrewAI, tareas y contexto secuencial: https://docs.crewai.com/en/concepts/tasks. CrewAI, contratos de herramientas: https://docs.crewai.com/en/concepts/tools. CrewAI, adaptadores de modelos: https://docs.crewai.com/en/concepts/llms. Referencias consultadas durante la implementación de EA2; versiones instaladas principales en requirements.txt y requirements-ea2.txt.

Guion sencillo para mostrar CAT-IA · EA2

Autoras: Antonella Cuvertino y Miriam Hammami. Docente: Sebastián Ignacio Sánchez Morales. Preparado el 11/10/2026.

Preparación

En este computador están instaladas las dependencias EA2. En otro equipo ejecutar instalar.bat y después instalar_ea2.bat para habilitar CrewAI. Para inferencia real iniciar_modelo_local.bat corresponde únicamente al equipo que tiene la instalación portable; luego iniciar.bat y abrir http://127.0.0.1:8000. No iniciar dos servidores en el mismo puerto.

Para un ensayo rápido sin modelo, configurar CATIA_PROVIDER=demo en .env y reiniciar la aplicación. Para el LLM real usar ollama y comprobar su URL. Leer el aviso de modo antes de hablar. El modo equipo en CPU puede demorar más por sus consultas adicionales. No afirmar que el simulador es IA generativa.

Diapositivas 1 y 2 · Antonella · 45 segundos

«Somos Antonella Cuvertino y Miriam Hammami. En esta segunda experiencia mejoramos CAT-IA, nuestro asistente de soporte para la empresa ficticia Nexo TI. Antes analizaba cada solicitud por separado. Ahora permite continuar una conversación y coordinar herramientas para preparar una respuesta con evidencia».

Diapositiva 3 · Antonella · 40 segundos

«Organizamos el trabajo en tres herramientas. Una busca documentos; otra consulta las reglas de prioridad; y otra genera el borrador y comprueba sus citas. El agente solo puede usar herramientas permitidas, por eso no puede enviar correos ni ejecutar acciones sobre clientes».

Si preguntan por código: catia/tools.py, create_tools y execute_tool. La prioridad usa impacto declarado; no basta escribir URGENTE.

Diapositiva 4 · Miriam · 40 segundos

«Cada conversación tiene su propio identificador. Conservamos los últimos tres intercambios y un resumen breve de entradas anteriores. Guardamos esto en SQLite para continuar después de reiniciar. El resumen es extractivo: recoge partes del texto, sin otra llamada de IA. Podemos borrar la memoria desde la pantalla».

Si preguntan por código: catia/memory.py, get, append y delete. La memoria no es entrenamiento y las respuestas anteriores no se convierten en documentos confiables.

Diapositiva 5 · Miriam · 45 segundos

«Tenemos dos modos. El individual selecciona una herramienta disponible y sigue un ciclo limitado. En el equipo, CrewAI organiza tres roles en secuencia: Triage revisa la prioridad, el Investigador recupera documentos y el Redactor clasifica y prepara el borrador. En ambos casos una persona debe revisar la respuesta».

Si preguntan por código: catia/orchestration.py, run_react y run_crew. El modo individual usa herramientas de LangChain; el equipo utiliza objetos Agent, Task, context y Process.sequential de CrewAI.

Diapositiva 6 · Demostración · 1 a 2 minutos más tiempo de inferencia

1. Abre la app. Di si estás usando demo o LLM real. Pulsa Borrar memoria y empezar para trabajar con un hilo limpio.

2. Selecciona Agente individual. Pulsa Cobro duplicado y después Analizar con evidencia. Muestra categoría, fuentes y Ver herramientas y agentes. El ejemplo desactiva contexto previo para no mezclar temas.

3. Activa Usar antecedentes de esta conversación. Escribe «¿Qué debo hacer en ese caso?» y vuelve a analizar. Muestra que el hilo conserva la consulta anterior sobre la factura. Verifica la respuesta, sin dar por correcto cualquier texto del modelo.

4. Abre Ver memoria de la conversación. Muestra los intercambios. Para enseñar el resumen, realiza más de tres consultas del mismo caso; evita hacerlo en vivo con LLM si el tiempo es corto. Se pueden preparar esos turnos antes o ensayarlos en demo.

5. Selecciona Equipo de 3 agentes. Envía de nuevo un caso claro. Muestra Triage, Investigador RAG y Redactor en las herramientas. Si ejecutas demo, explica que CrewAI sí se ejecuta, pero las decisiones del modelo son simuladas.

6. Pulsa Borrar memoria y empezar. Comprueba que desaparece el historial del hilo. Inicia un caso nuevo. No prometas que una pregunta genérica siempre se abstendrá: la similitud puede recuperar algún documento poco pertinente.

Diapositiva 7 · Pruebas · 40 segundos

«Agregamos pruebas para comprobar que una sesión no use datos de otra, que la memoria persista, que se pueda borrar y que las herramientas rechacen acciones no permitidas. También distinguimos los ensayos con simulador de las pruebas reales con Ollama. Los resultados actuales están en el informe de EA2».

Para ejecutarlas: .\.venv\Scripts\python.exe -m pytest -q. Para evidencia detallada abrir reports/EA2_RESULTADOS.md. Leer el resultado real: no memorizar un número si luego cambia la suite.

Diapositiva 8 · Cierre · 30 segundos

«El aporte de esta etapa es dar continuidad a los casos y hacer visible cómo colaboran memoria, herramientas y agentes. La aplicación sigue siendo un prototipo local. Las citas se comprueban, pero todavía necesitamos revisar si cada recomendación es correcta y pertinente antes de usarla».

Si falla el modelo

Sin la casilla de respaldo se muestra un error controlado. Si habilitas Permitir simulación si falla el proveedor, la pantalla indica SIMULACIÓN DE RESPALDO cuando se utiliza. Di «el modelo falló; ahora mostramos el flujo simulado». No atribuyas ese borrador al LLM ni a una ejecución real del equipo.

Tres respuestas cortas para practicar

¿Entrenaron una IA? «No modificamos los pesos del modelo. Adaptamos documentos, instrucciones, herramientas y memoria».

¿La memoria y el RAG son lo mismo? «No. La memoria conserva antecedentes del caso; el RAG busca documentos que respaldan la respuesta».

¿El sistema decide solo? «Decide dentro de un catálogo y límites programados. La prioridad sigue reglas y la persona revisa el borrador antes de usarlo».

Resultados verificables · EA2

Autoras: Antonella Cuvertino y Miriam Hammami. Actualización: 11/10/2026.

Software

45 pruebas: 0 fallos y 0 omitidas. Cobertura de líneas: 93.8 %. Evidencia: reports/tests_ea2.xml y reports/coverage_ea2.json.

Incluyen las pruebas anteriores y las de continuidad, aislamiento, persistencia, borrado, concurrencia local, herramientas, API, errores y CrewAI real con generación simulada.

Integración por proveedor

- demo / react: PASA; estados draft, draft; duración por caso 0.2 s, 0.1 s. Fuente: ea2_demo_react.json; fecha 2026-10-11T15:11:39.025996+00:00.
- ollama / crew: PASA; estados draft; duración por caso 156.9 s. Fuente: ea2_ollama_crew.json; fecha 2026-10-11T22:28:03.216670+00:00.
- ollama / react: PASA; estados draft, draft; duración por caso 117.0 s, 104.5 s. Fuente: ea2_ollama_react.json; fecha 2026-10-11T15:07:15.105966+00:00.

Interfaz

Resultado: PASA. Continuación de facturación con antecedentes y memoria visible; CrewAI simulado: tres herramientas y fuentes internas/externas visibles; Pantalla móvil de 390 px sin desbordamiento horizontal; Borrado desde la interfaz: memoria vacía y sesión devuelve 404. Evidencia: reports/browser_checks_ea2.json.

Alcance y límites

Los casos de integración son sintéticos y pequeños. No constituyen una evaluación independiente de fidelidad, relevancia o generalización. La validación de citas comprueba procedencia y coincidencia textual; la revisión semántica sigue siendo humana.

Las métricas de EA1 pertenecen a su implementación y prompt v3; no se transfieren al agente EA2 con prompt v4. La cobertura no mide calidad del LLM. Las latencias dependen del hardware y la carga.

Se observaron advertencias de deprecación en dependencias CrewAI/Pydantic durante las pruebas. No impidieron completarlas; revisar al actualizar versiones.